Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410048	26102000170	BWU/MCA/26/053	Somnath Chatterjee

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

Campus Event Management & Feedback Analyzer

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410048	26102000170	BWU/MCA/26/053	Somnath Chatterjee

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[COMPUTATIONAL SCIENCES]

BONAFIDE CERTIFICATE

Certified that this project report **"Campus Event Management & Feedback Analyzer"** is the bonafide work **of "Somnath Chatterjee"** who carried out the project work under my supervision.

_ _ _ _ _ _ _ _ _ _ _	_ _ _ _ _ _ _ _ _ _ _
SIGNATURE	**SIGNATURE**
DR. Jayanta Aich	MR. Rahul Kumar Ghosh
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Computational Sciences	Assistant Professor
Brainware University	Computational Sciences

ABSTRACT

This project presents a Python-based Streamlit application that manages campus events and analyzes participant feedback data from CSV files. The system reads event records, student registrations, and related data, calculates key metrics such as registration counts, attendance rates, average ratings, and event popularity rankings, and generates interactive visual charts for better insights. It is developed primarily in a single application file (main.py) supported by sample CSV datasets (events.csv, registrations.csv, students.csv) and relies on libraries such as pandas, numpy, plotly, and Streamlit.

A clean web-based interface allows users to load data, explore events, view registration statistics, analyze feedback ratings, and generate graphical reports without complex coding. The system identifies popular events, low-rated sessions, registration trends, and produces clear interactive visualizations. This reduces manual tracking effort for event organizers and administrators and demonstrates how Python data analysis and interactive visualization techniques can be combined into a practical campus event management tool.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Data Loading	15
7.2	Event Statistics	15
7.3	Registration Analysis	16
7.4	Feedback Rating Charts	16
7.5	Interactive Plotly Visualizations	17
7.6	Event Popularity Ranking	17

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for event & registration data input)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface (Streamlit web interface)
Avg	Average (used for average ratings and attendance)
ID	Identifier (Event ID / Student ID)
□	Data flow / Message direction (in DFD and Sequence Diagram)
■ / ■	Process / Activity symbol (in Flow Chart and Activity Diagram)
■	Decision symbol (in Flow Chart and Activity Diagram)
■ (open)	Data Store (in DFD)
Actor	External entity / User (in Use Case and Context diagrams)

CHAPTER 1: INTRODUCTION

Traditional campus event management relies on manual spreadsheets, paper registration forms, and scattered feedback collection. Organizers often struggle to quickly identify popular events, track registration trends, or extract meaningful insights from participant feedback, and generating summary reports or visual insights requires additional effort. With the availability of Python data analysis libraries and modern web frameworks such as Streamlit, it is now possible to automate large parts of this process.

This project focuses on building a system that reads event data, student registrations, and feedback from CSV files, processes the records, calculates important metrics, and produces both textual statistics and interactive graphical charts. The idea is simple: the user provides structured data files, and the program computes registration counts, attendance rates, average ratings, event popularity rankings, and campus-level summaries, then visualizes the results through an interactive dashboard.

The system uses a Streamlit-based design with a main application file supported by sample CSV datasets. Behind the interface, the workflow follows clear steps: load data, validate records, compute event and feedback metrics, generate interactive visualizations, and present a consolidated dashboard. The project demonstrates how data processing, statistical calculation, and interactive visualization can be combined into a lightweight, easy-to-understand campus event analytics tool suitable for students, organizers, and beginners.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python-based Campus Event Management & Feedback Analyzer that enables users to process event and registration records from CSV files and obtain meaningful participation and feedback insights. The system aims to provide an interactive, automated workflow by integrating data loading, metric calculation, and visualization components. Users can load event and registration data, compute individual and campus-level statistics (registration counts, attendance percentages, average ratings, event rankings), identify high-performing or under-attended events, and generate interactive charts without requiring advanced programming knowledge. By combining pandas for data handling, custom analysis functions, Plotly-based interactive visualizations, and a Streamlit web interface, the project intends to simplify event monitoring, reduce manual calculation effort, and offer an accessible end-to-end event management and feedback analysis pipeline for campus use.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Campus Event Management & Feedback Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to analyze campus events and feedback from CSV data.
- Identify the target users (event organizers, students, academic administrators, beginners).
- Define the core requirement: CSV data → analysis of events & feedback → statistics + interactive charts.

2. System Design Planning

- Decide to use pandas for data handling, Plotly for interactive visualization, and Streamlit for the web interface.
- Choose a single-file application structure (main.py) with supporting CSV datasets.
- Plan the workflow from data loading to final interactive dashboard.
- Select supporting libraries: streamlit, pandas, numpy, plotly.

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from events.csv, registrations.csv, and students.csv
- Data validation and cleaning
- Event and registration metric calculation
- Feedback rating analysis and ranking
- Statistical summaries
- Interactive chart generation
- Dashboard display

4. File & Module Planning

- main.py – Streamlit entry point and full application logic
- events.csv – Sample event data
- registrations.csv – Sample registration records
- students.csv – Sample student master data

5. Code Structure Planning

Break the code into clear, reusable sections:

- Data loading and validation from multiple CSVs
- Metric calculation functions (registrations per event, average ratings, attendance rates)
- Filtering of popular / low-rated events
- Interactive Plotly chart plotting functions
- Streamlit sidebar navigation and main dashboard layout

6. Testing & Validation Plan

- Test with sample CSV data.
- Verify correct registration counts, rating averages, and ranking calculations.
- Check interactive chart generation for different metrics.
- Validate handling of edge cases (missing data, empty feedback, zero registrations).

7. Scalability & Improvement Planning

Plan for expandable features such as:

- Support for multi-campus or multi-semester data
- Export of reports to PDF or Excel
- Advanced sentiment analysis on textual feedback
- Predictive analysis using simple machine learning
- Database integration instead of CSV
- User authentication and role-based access

CHAPTER 4: REQUIREMENT ANALYSIS

Functional Requirements

1. Data Input

- The system must read event, registration, and student records from CSV files.
- Each record typically includes event identifiers, registration status, ratings, and related information.
- Input is handled through Python file reading and pandas DataFrame operations.

2. User Interaction

- A Streamlit web interface provides sidebar navigation and interactive dashboard views.
- Options to load data, explore events, view statistics, generate charts, and filter results.
- Users should not need to write code to obtain results.

3. Model and Framework Requirements

The project depends on the following Python libraries:

- pandas (data loading and manipulation)
- numpy (numerical operations)
- plotly / plotly.express (interactive chart generation)
- streamlit (web interface and dashboard)
- Custom application logic contained in main.py

4. Analysis Requirements

For every event / registration set:

- Calculate registration counts and attendance rates.
- Calculate average feedback ratings.
- Rank events by popularity or rating.
- Identify events with low attendance or poor feedback.

5. Visualization Requirement

- Generate interactive bar charts, pie charts, histograms, and ranking views.
- Charts should be displayed clearly within the Streamlit dashboard.

6. Output Organization

- Display textual summaries and statistics on the dashboard.
- Generate interactive charts that support exploration and decision-making.
- Provide a clear consolidated view of campus-level and event-level insights.

CHAPTER 5: SYSTEM FLOW

Context Level Diagram (CLD):

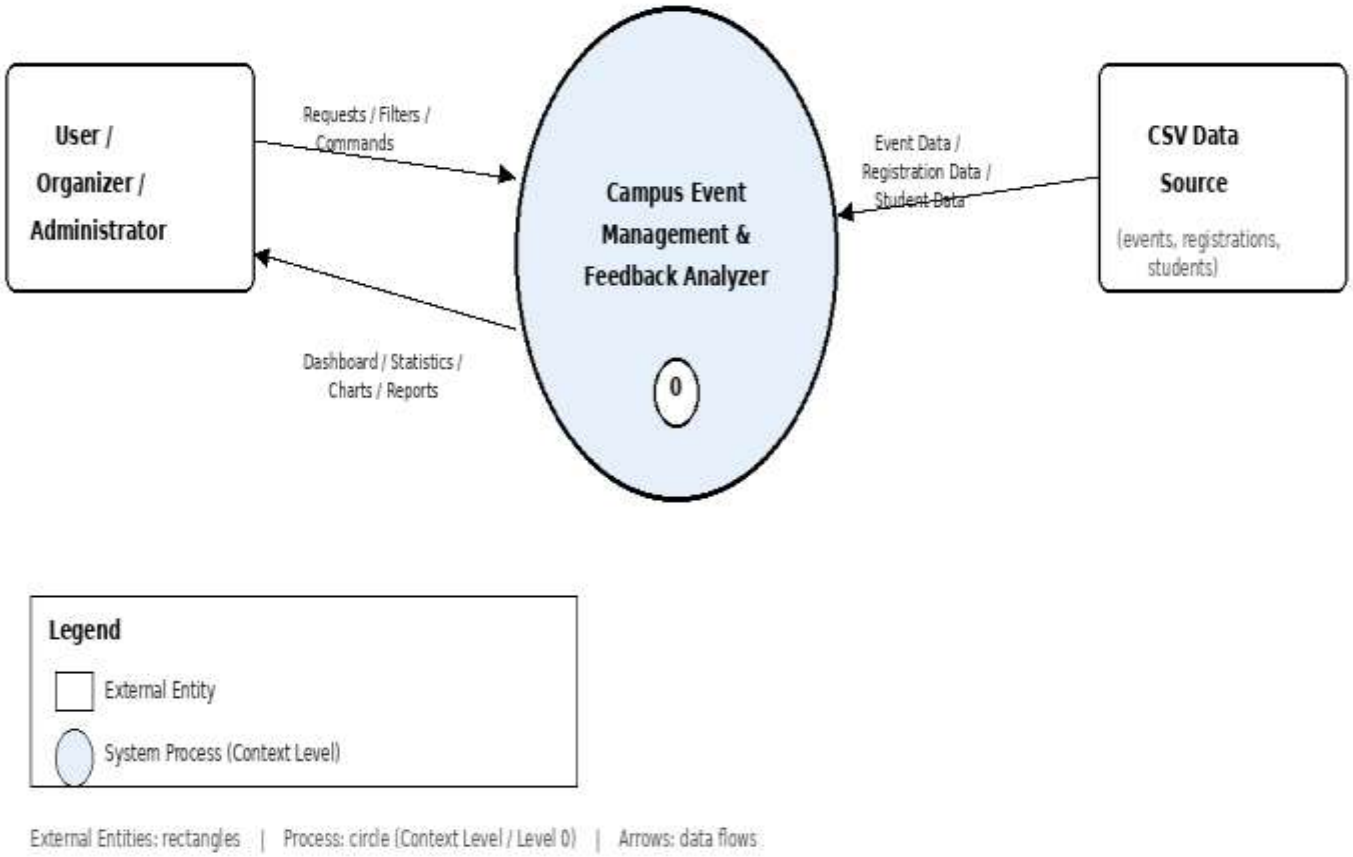


Fig 5.1: Context Level Diagram

It shows the overall system as a single process interacting with external entities (User / Organizer and CSV data sources) and producing interactive dashboards, statistics, and charts.

Data Flow Diagrams (DFD):

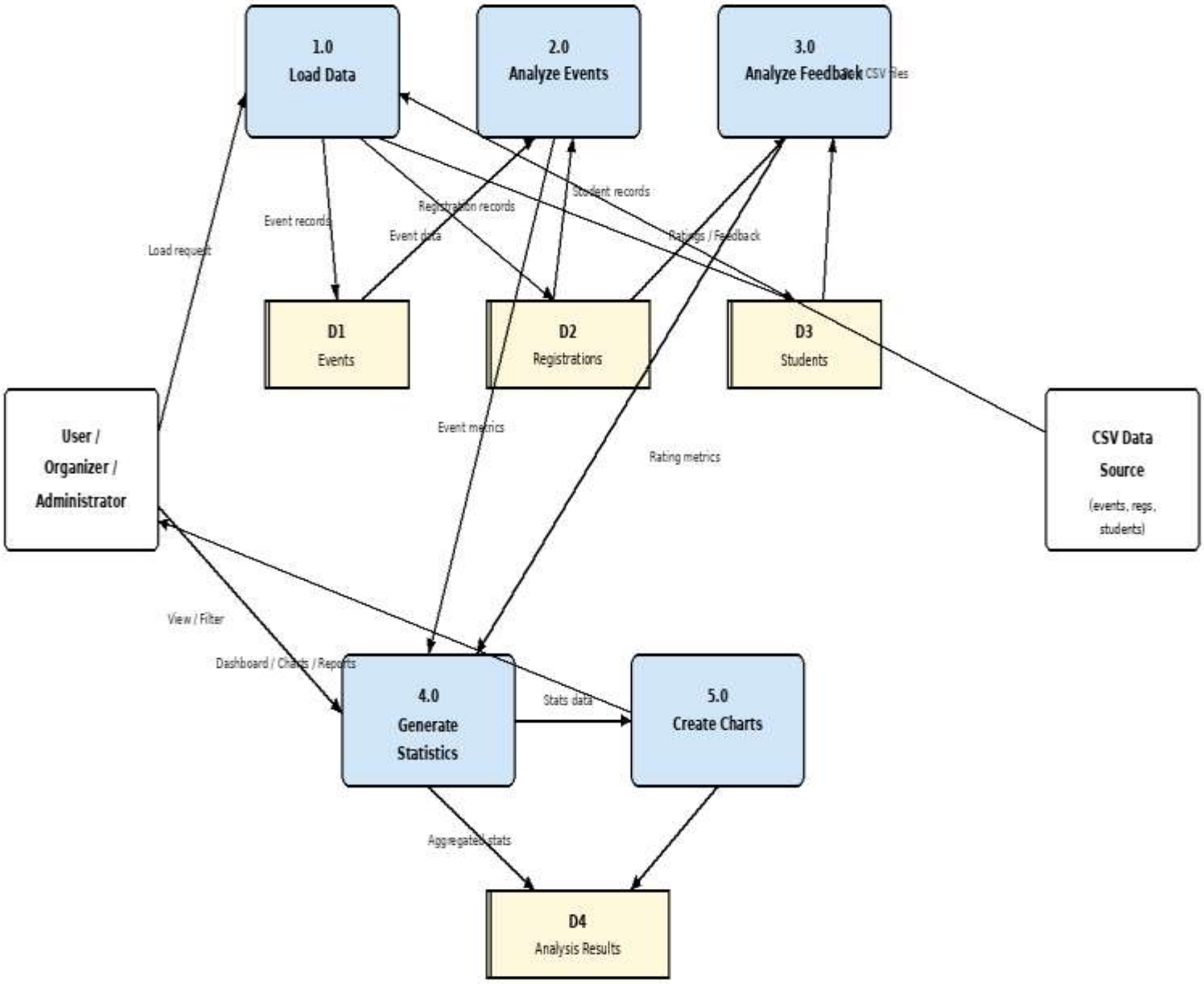


Fig 5.2: Data Flow Diagram Level 0

It illustrates the major processes (Load Data, Analyze Events, Analyze Feedback, Generate Statistics, Create Charts) along with data stores and high-level data flow.

Data Flow Diagrams (DFD):

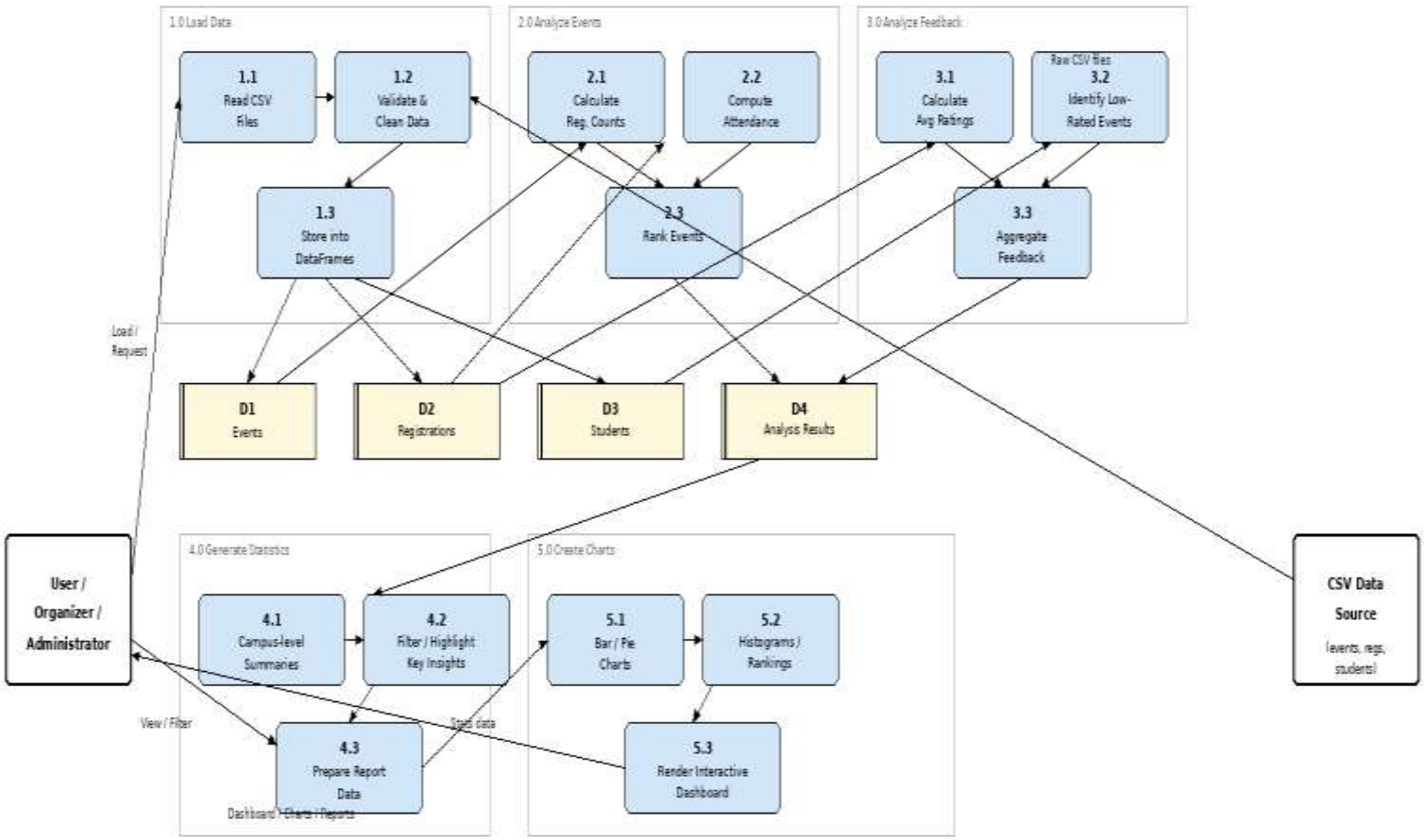


Fig 5.3: Data Flow Diagram Level 1

It breaks down the Level 0 processes into detailed sub-processes for data loading, metric calculation, statistics generation, ranking, and chart creation.

Use Case Diagram:

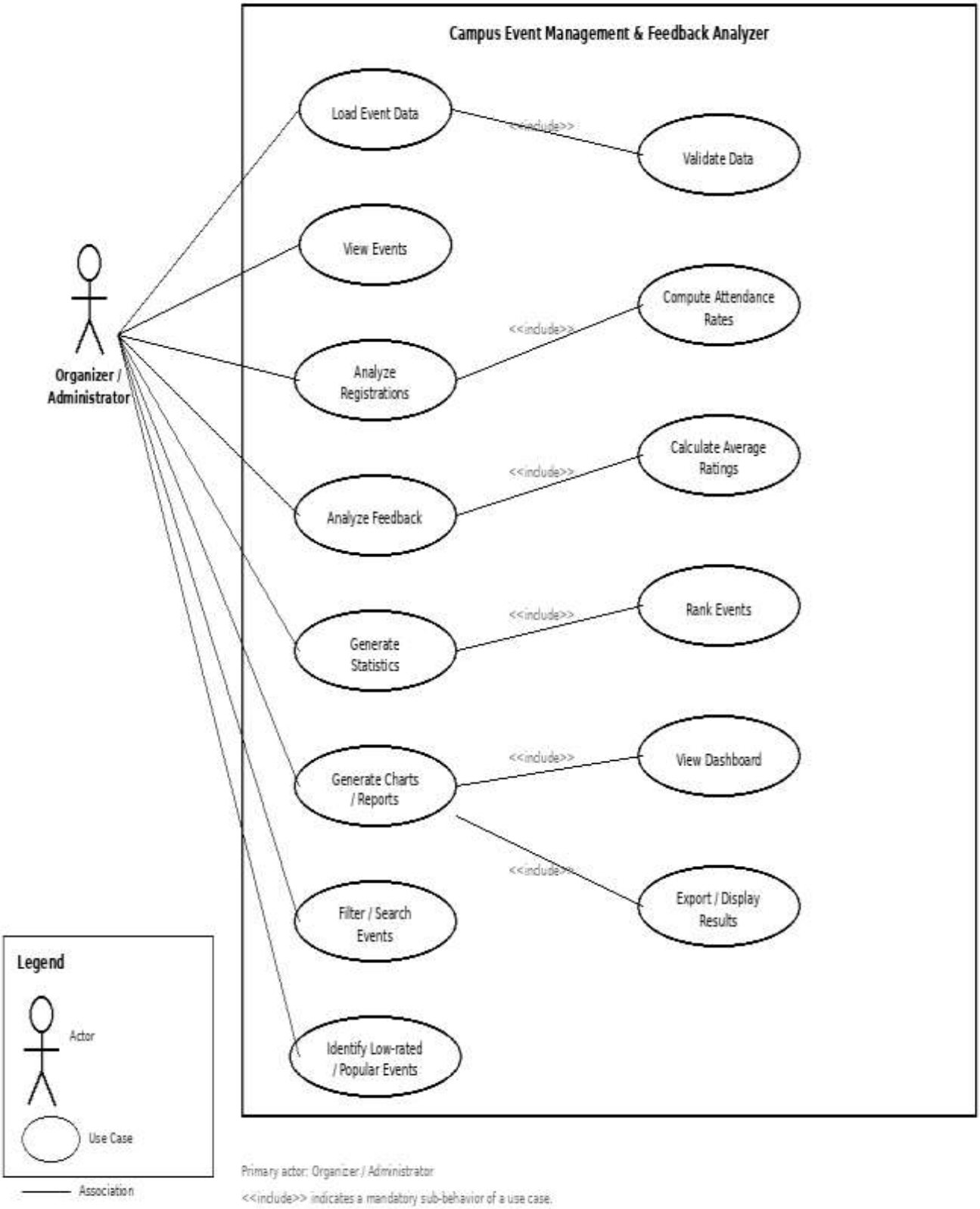


Fig 5.4: Use Case Diagram

It depicts the interactions between the Organizer / Administrator and the system's key functionalities such as loading data, analyzing events and feedback, and generating reports / dashboards.

Activity Diagram:

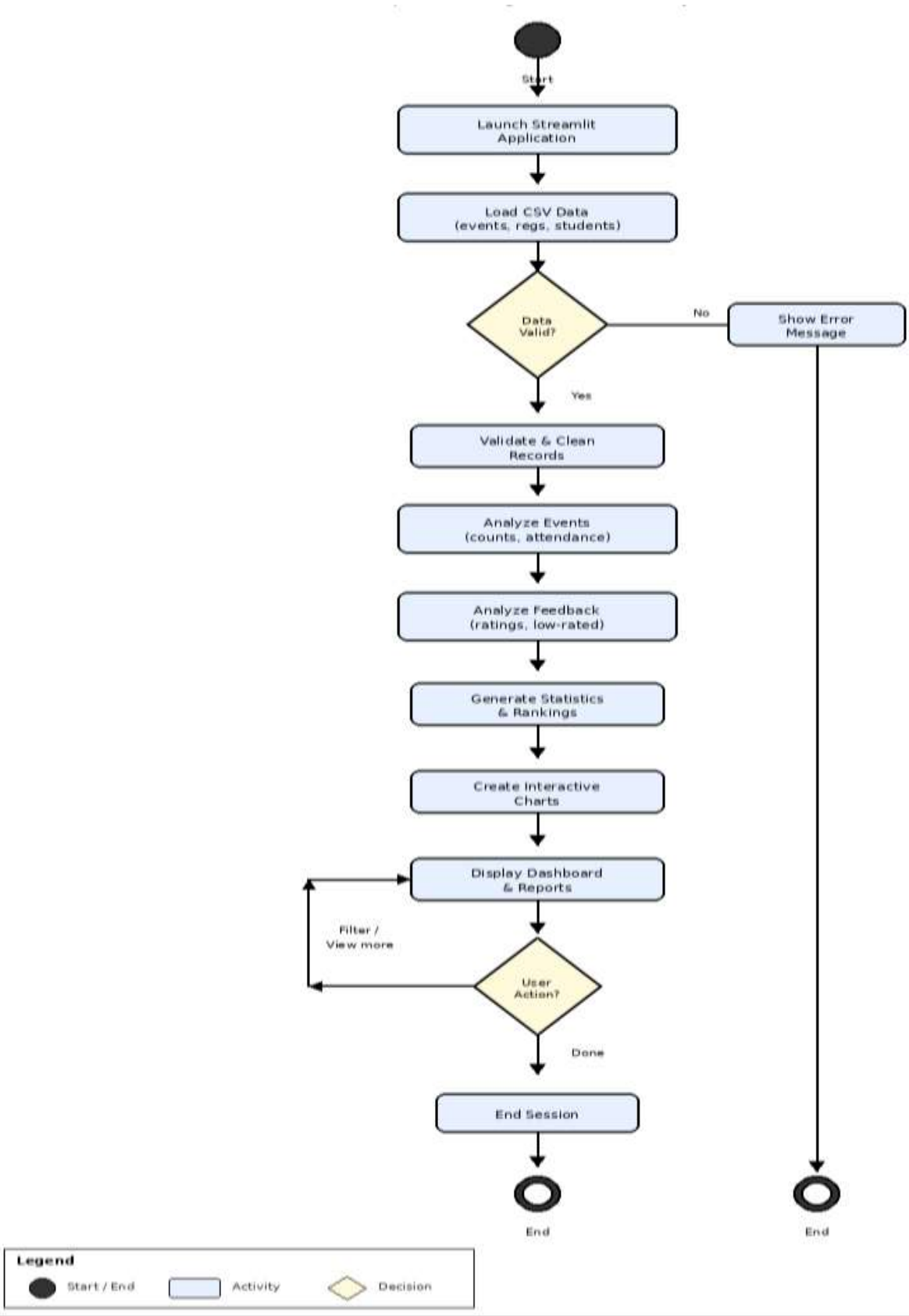


Fig 5.5: Activity Diagram

It represents the step-by-step workflow of the system from starting the Streamlit application to displaying the final interactive dashboard, including decision points for data validation.

Sequence Diagram:

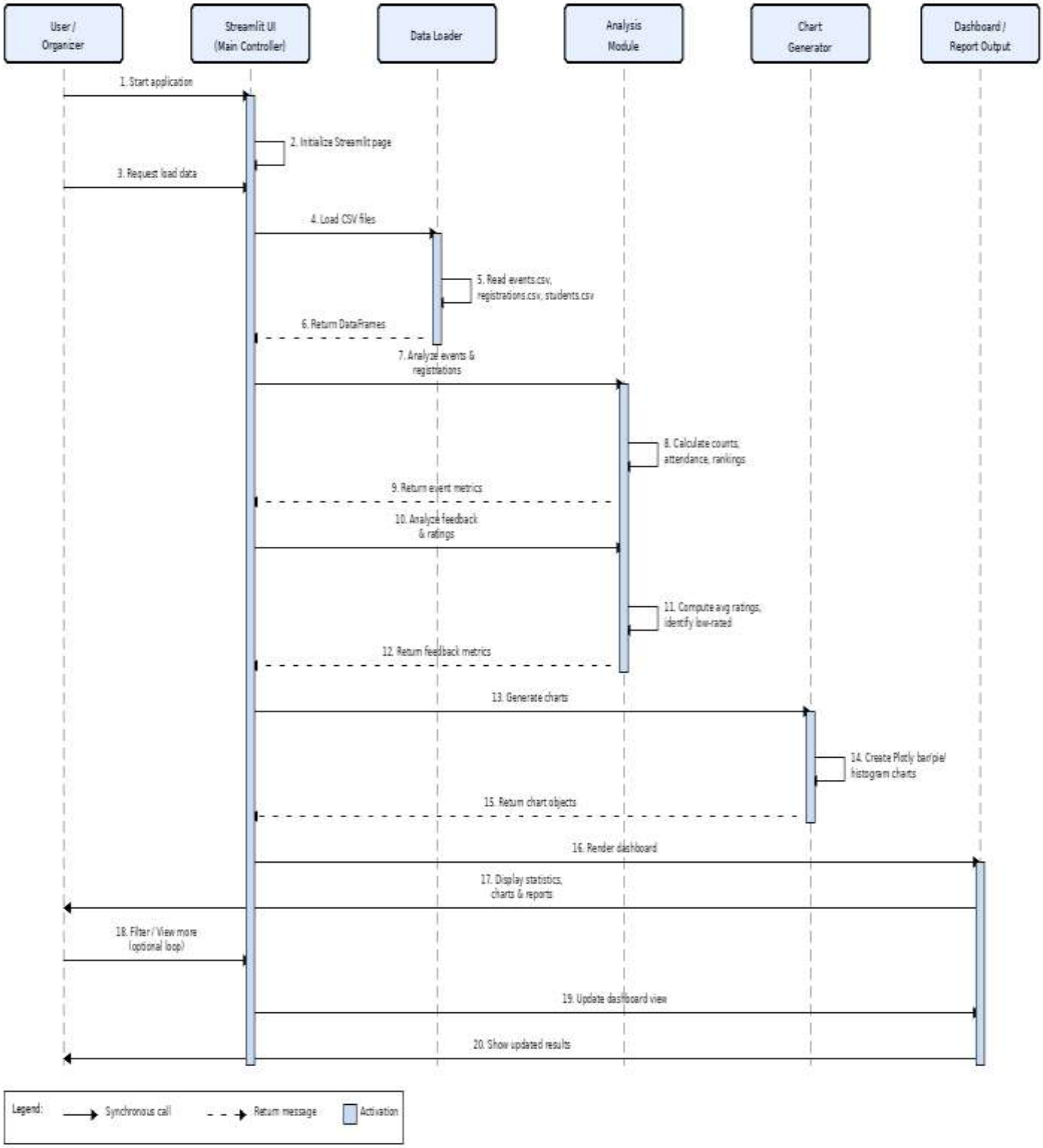


Fig 5.6: Sequence Diagram

It shows the chronological sequence of messages exchanged between the User, Streamlit Controller, Data Loader, Analysis Module, Chart Generator, and Dashboard Output.

Flow Chart:

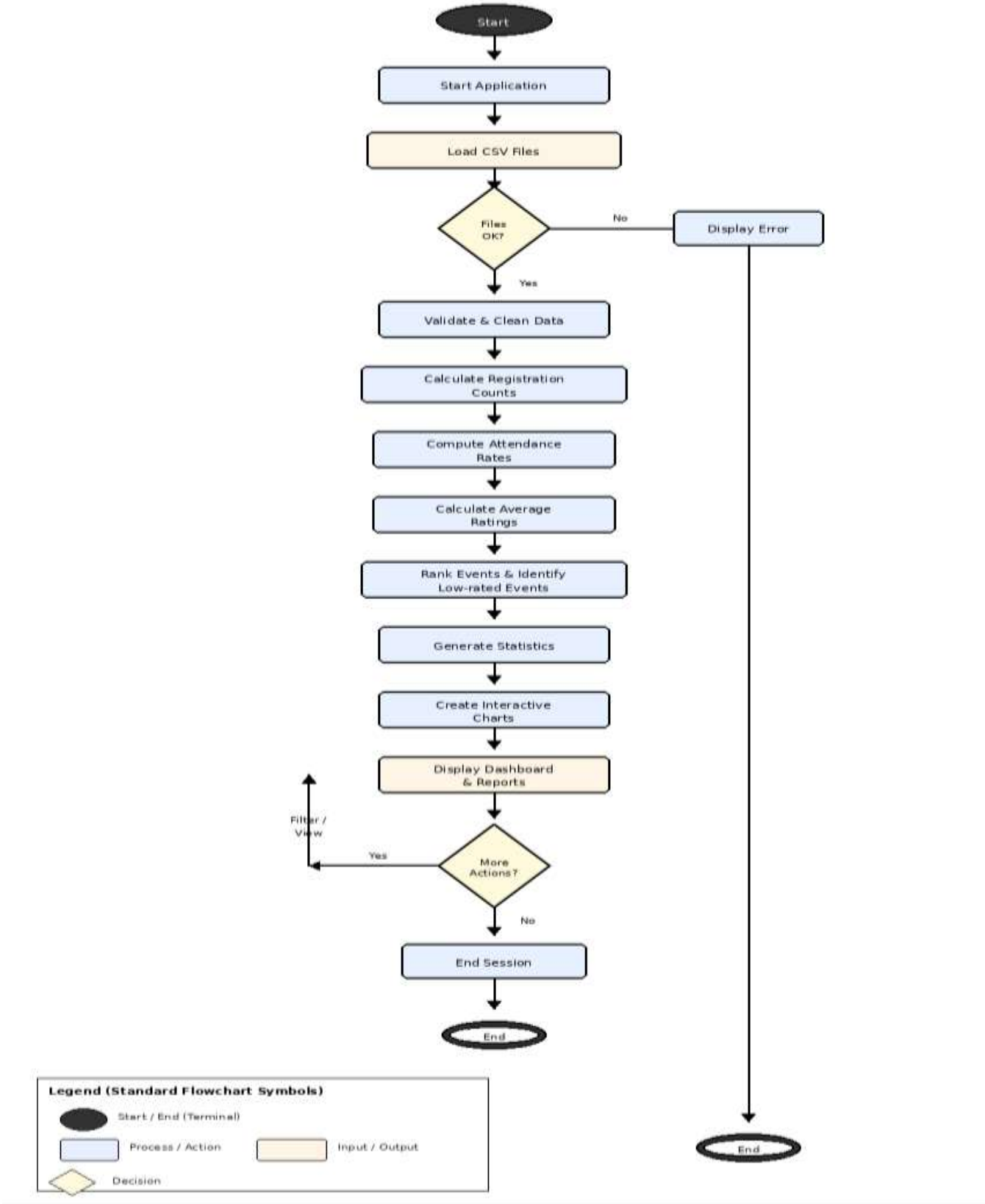


Fig 5.7: Flow Chart

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for data validation and the path to final dashboard generation.

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes event and registration data and produces analysis results and interactive charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations. The overall workflow is shown in the diagrams, and each stage is described in detail.

The application loads event records, registrations, and student data from CSV files, creates structured DataFrames, computes performance and feedback metrics, ranks events, and generates interactive charts. This method allows users to obtain insights from campus event data without manual calculations or complex tools.

The process includes the following key stages:

1. Data loading from CSV
2. Data modeling / DataFrame preparation
3. Event and registration analysis
4. Feedback rating analysis and ranking
5. Interactive chart creation and display
6. Final dashboard

presentation The steps included

are –

Step 1: Data Loading

The program reads the events.csv, registrations.csv, and students.csv files using pandas and prepares clean DataFrames for analysis.

Step 2: Data Modeling

Structured records with attributes such as event ID, name, category, date, student ID, registration status, and feedback rating are prepared for further processing.

Step 3: Analysis

The analysis logic performs:

- Calculation of registration counts per event
- Attendance rate computation
- Average feedback rating calculation
- Ranking of events by popularity or rating
- Identification of events needing attention
- Campus-level aggregates (overall averages, top/bottom events)

Step 4: Chart Generation

The application uses Plotly to produce interactive visual representations such as:

- Registration count bar charts
- Feedback rating distribution (histograms / pie charts)
- Event popularity ranking views
- Category-wise or trend comparisons when applicable

Step 5: Main Program Flow

main.py (Streamlit) orchestrates the process: loads data, performs analysis, triggers chart generation, and presents results through a sidebar-driven interactive dashboard.

Step 6: Final Output

Users receive dashboard statistics, lists of popular or low-rated events, and interactive charts. This supports both verification and decision-making.

This methodology is effective because:

- Data processing is automated and repeatable
- Streamlit design makes the tool easy to run and extend
- Interactive visualizations improve insight communication
- No external specialized software is required beyond standard Python libraries
- The workflow is suitable for undergraduate-level implementation while demonstrating practical data analysis techniques

REPRESENTATIVE CODE STRUCTURE (based on project files)

- **main.py**: Streamlit entry point containing page configuration, data loading, analysis, and all dashboard / chart rendering logic.
- **events.csv**: Sample dataset of campus events.
- **registrations.csv**: Sample student registration records linked to events.
- **students.csv**: Sample student master data.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how event and feedback metrics were calculated, how statistics were produced, and how interactive charts were generated. Screenshots of the Streamlit dashboard, charts, and sample reports can be inserted in the respective places.

1. Data Loading Stage

The system successfully reads the events.csv, registrations.csv, and students.csv files and prepares the data for analysis.

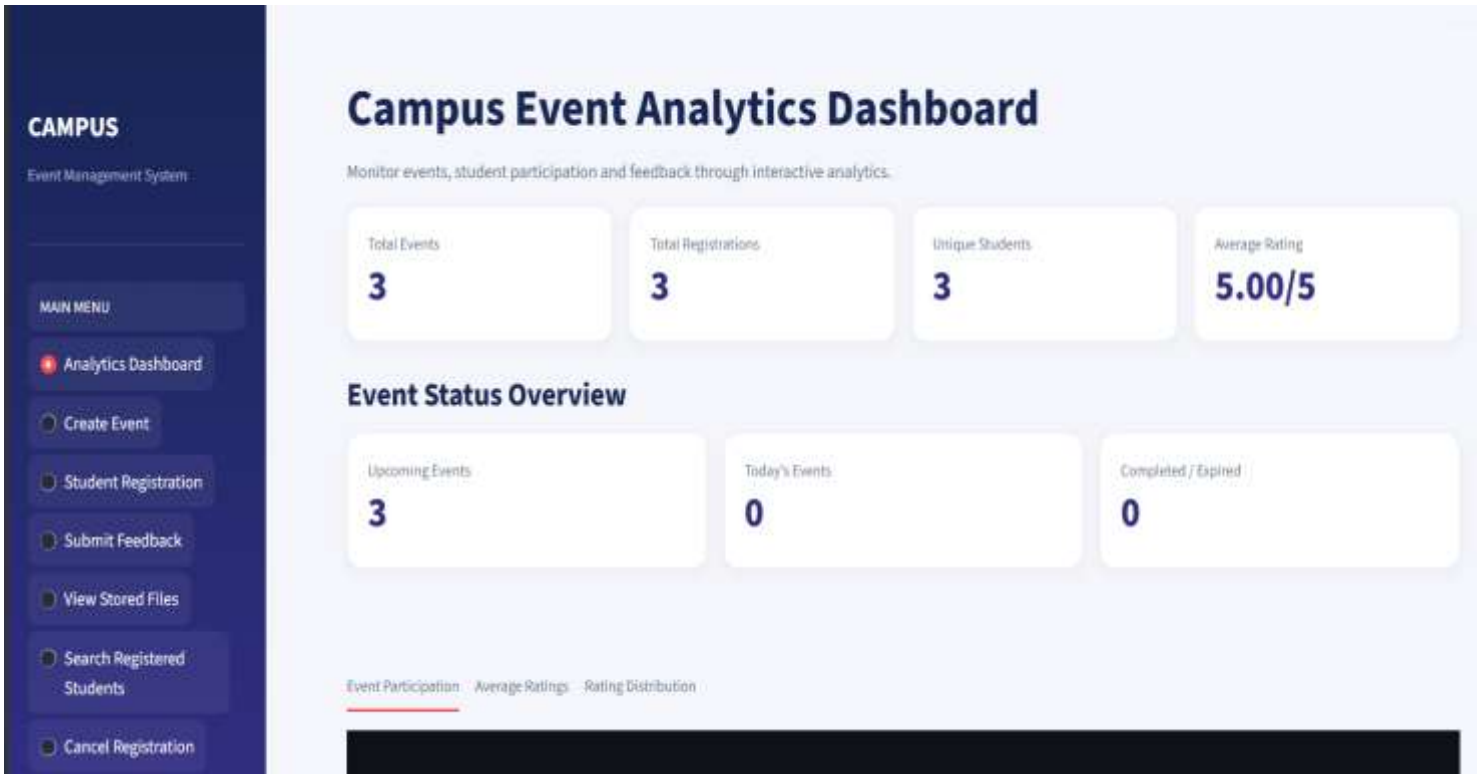


Fig 7.1: Data Loading Stage

2. Event & Registration Analysis

Registration counts, attendance rates, and event rankings are calculated for each event and for the campus overall.

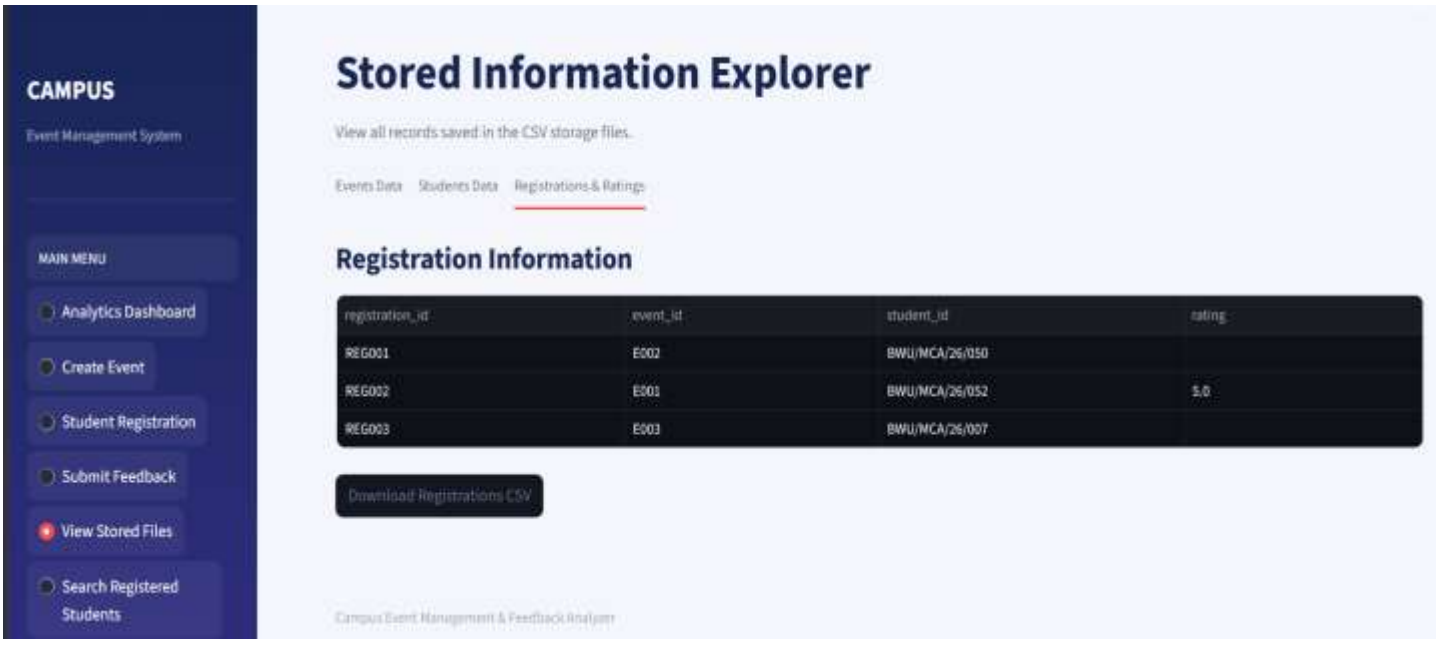


Fig 7.2: Event Statistics

3. Feedback Analysis

Average ratings are computed, rating distributions are generated, and events falling below a quality threshold are identified.



Fig 7.3: Registration / Feedback Analysis

4. Charts Generation

Interactive Plotly charts (bar, pie, histogram, ranking) are produced for registration counts, rating distributions, and performance comparison.



Fig 7.4: Feedback Rating Charts

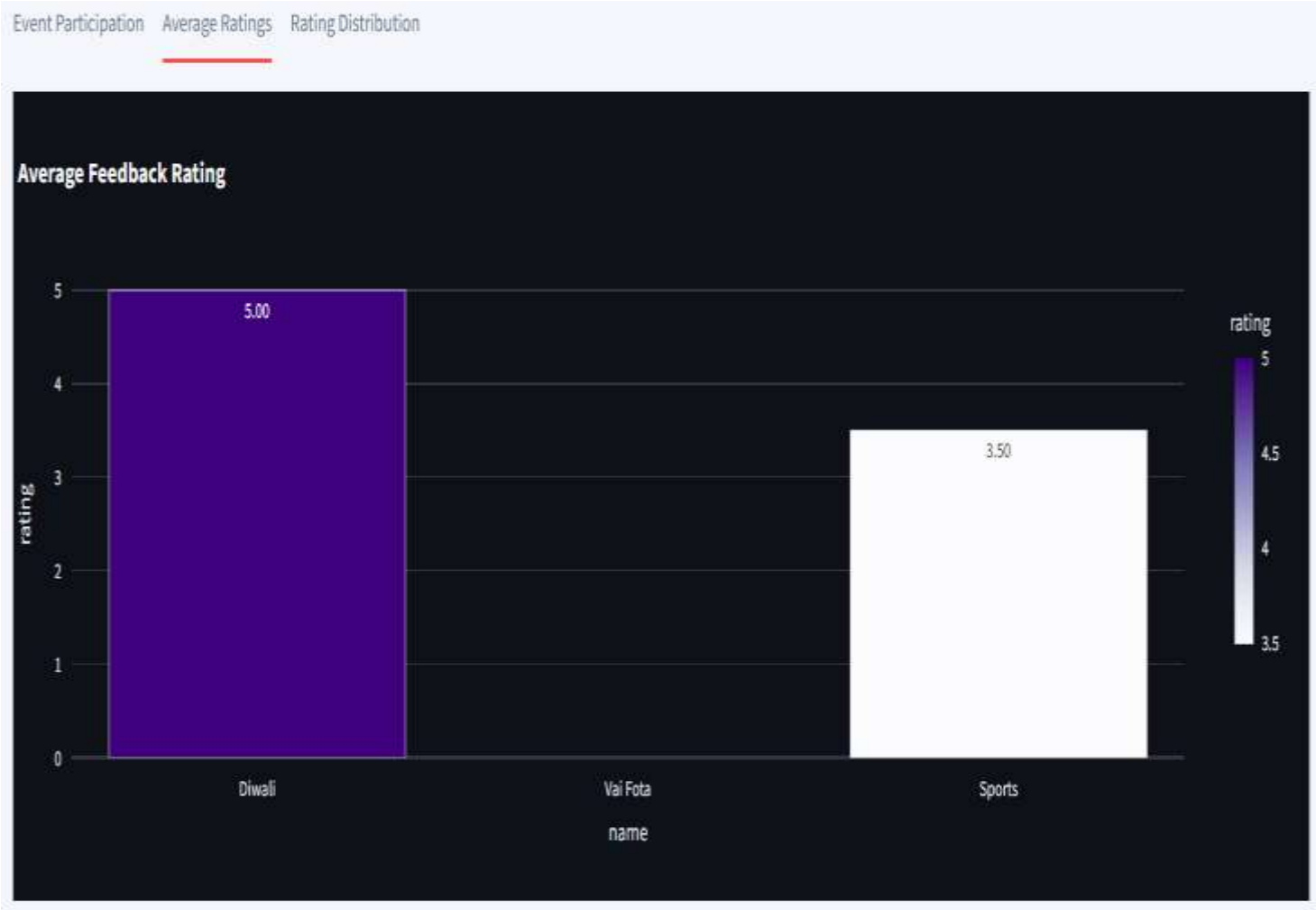


Fig 7.5: Interactive Plotly Visualizations

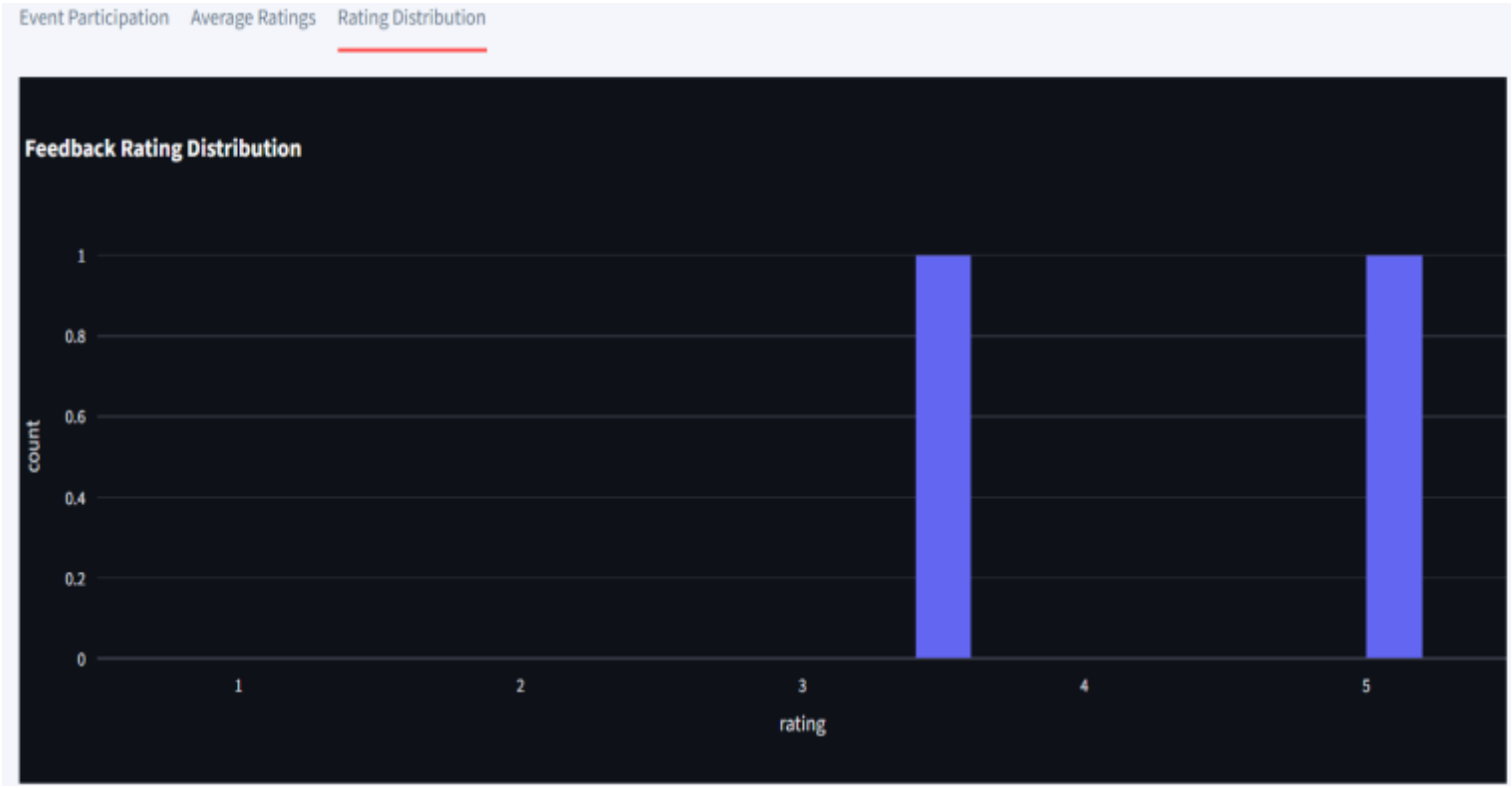


Fig 7.6: Event Popularity Ranking

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. Enhanced User Interface

Add user authentication, role-based dashboards (Admin / Organizer / Student), and richer filtering controls for easier data exploration and real-time visualization.

2. Multi-Campus & Multi-Semester Support

Extend the data model to handle multiple campuses, continuous assessment components, and semester-wise tracking.

3. Report Export Features

Add options to export analysis results and charts to PDF, Excel, or PowerPoint for easy sharing with administrators and stakeholders.

4. Advanced Feedback Analytics

Incorporate NLP-based sentiment analysis and keyword extraction on textual feedback comments to provide deeper qualitative insights.

5. Database Integration

Replace or supplement the CSV input with a proper database (SQLite / MySQL / PostgreSQL) for larger datasets and multi-user access.

6. Notification & Alert System

Add automatic alerts or email notifications for low-rated events, capacity warnings, or registration deadlines.

7. Predictive Analytics

Incorporate simple machine learning models to predict attendance or identify at-risk events based on historical patterns.

CHAPTER 9: CONCLUSION

The project successfully showcases a complete, end-to-end workflow for managing campus events and analyzing participant feedback from structured data through an automated and interactive Python pipeline. Beginning with CSV input, the system efficiently loads records, computes meaningful metrics (registration counts, attendance rates, average ratings, event rankings), identifies events requiring attention, and generates clear interactive visualizations. This demonstrates that practical campus event analytics can be achieved using standard Python libraries (pandas, numpy, plotly) and Streamlit when the process is divided into clear stages.

Moreover, the Streamlit-based design significantly enhances usability, allowing individuals with basic Python knowledge to understand, run, and extend the system. The design emphasizes accessibility and practicality, ensuring that event monitoring and feedback analysis is not limited to complex enterprise tools. Overall, the project meets its primary objective by delivering a functional, data-driven Campus Event Management & Feedback Analyzer that is both practical and educationally valuable.

REFERENCES

1. pandas Development Team. pandas: Powerful data structures for data analysis. Available at: https://pandas.pydata.org/

2. Plotly Technologies Inc. Plotly Python Open Source Graphing Library. Available at: https://plotly.com/python/

3. Streamlit Inc. Streamlit – A faster way to build and share data apps. Available at: https://streamlit.io/

4. NumPy Developers. NumPy. Available at: https://numpy.org/

5. Python Software Foundation. Python Language Reference. Available at: https://www.python.org/

6. Project source code and files. Available at: https://1drv.ms/f/c/0ca0c46877729f6d/IgC4al2PlEgOR5drAZcpAz6eAVhNEeLKEkRh2AjnDBgVkBA?e=7Nqcid